

```
#Problem Statement:Binary classification using Deep Neural Networks Example:  
# Classify movie reviews into positive" reviews and "negative" reviews,  
# just based on the text content of the reviews. Use IMDB dataset.
```

```
!pip install tensorflow
```

```
Requirement already satisfied: wheel<1.0,>=0.23.0 in  
c:\users\dell-pc\anaconda3\lib\site-packages (from  
astunparse>=1.6.0->tensorflow) (0.45.1)  
Requirement already satisfied: rich in  
c:\users\dell-pc\anaconda3\lib\site-packages (from keras>=3.12.0->tensorflow)  
(14.2.0)  
Requirement already satisfied: namex in  
c:\users\dell-pc\anaconda3\lib\site-packages (from keras>=3.12.0->tensorflow)  
(0.1.0)  
Requirement already satisfied: optree in  
c:\users\dell-pc\anaconda3\lib\site-packages (from keras>=3.12.0->tensorflow)  
(0.19.0)  
Requirement already satisfied: markdown-it-py>=2.2.0 in  
c:\users\dell-pc\anaconda3\lib\site-packages (from  
rich->keras>=3.12.0->tensorflow) (2.2.0)  
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in  
c:\users\dell-pc\anaconda3\lib\site-packages (from  
rich->keras>=3.12.0->tensorflow) (2.19.2)  
Requirement already satisfied: mdurl~=0.1 in  
c:\users\dell-pc\anaconda3\lib\site-packages (from  
markdown-it-py>=2.2.0->rich->keras>=3.12.0->tensorflow) (0.1.2)
```

```
import sys  
print(sys.executable)  
import tensorflow as tf  
print(tf.__version__)
```

```
C:\Users\DELL-PC\anaconda3\python.exe  
2.21.0
```

```
from tensorflow.keras.datasets import imdb
```

```
(train_data, train_labels), (test_data, test_labels) =  
imdb.load_data(num_words=10000)
```

```
Downloading data from  
https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz  
17464789/17464789 ----- 74s 4us/step
```

```
train_data[0]
```

```
[1,  
 14,  
 22,  
 16,
```

```
43,  
530,  
973,  
1622,  
1385,
```

```
4472,  
113,  
103,  
32,  
15,  
16,  
5345,  
19,  
178,  
32]
```

```
train_labels[0]
```

```
np.int64(1)
```

```
print(type([max(sequence) for sequence in train_data]))  
max([max(sequence) for sequence in train_data])
```

```
<class 'list'>
```

```
9999
```

```
import ssl  
ssl._create_default_https_context = ssl._create_unverified_context  
word_index = imdb.get_word_index()  
reverse_word_index = dict([(value, key) for (key, value) in  
word_index.items()])  
decoded_review = ''.join([reverse_word_index.get(i-3, '?') for i in  
train_data[0]])  
decoded_review
```

```
Downloading data from
```

```
https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb\_word\_index.j  
son
```

```
1641221/1641221 ————— 5s 3us/step
```

```
"? this film was just brilliant casting location scenery story direction  
everyone's really suited the part they played and you could just imagine  
being there robert ? is an amazing actor and now the same being director ?  
father came from the same scottish island as myself so i loved the fact there  
was a real connection with this film the witty remarks throughout the film  
were great it was just brilliant so much that i bought the film as soon as it  
was released for ? and would recommend it to everyone to watch and the fly  
fishing was amazing really cried at the end it was so sad and you know what
```

they say if you cry at a film it must have been good and this definitely was also ? to the two little boy's that played the ? of norman and paul they were just brilliant children are often left out of the ? list i think because the stars that play them all grown up are such a big profile for the whole film but these children are amazing and should be praised for what they have done don't you think the whole story was so lovely because it was true and was someone's life after all that was shared with us all"

```
len(reverse_word_index)
```

```
88584
```

```
import numpy as np
def vectorize_sequences(sequences, dimension = 10000):
    results = np.zeros((len(sequences),dimension))
    for i,sequence in enumerate(sequences):
        results[i,sequence] = 1
    return results
```

```
xtrain = vectorize_sequences(train_data)
xtest = vectorize_sequences(test_data)
```

```
xtrain[0]
```

```
array([0., 1., 1., ..., 0., 0., 0.], shape=(10000,))
```

```
xtrain.shape
```

```
(25000, 10000)
```

```
ytrain = np.asarray(train_labels).astype('float32')
ytest = np.asarray(test_labels).astype('float32')
```

```
from tensorflow.keras import models
from tensorflow.keras import layers
```

```
model = models.Sequential()
```

```
#  Add Input Layer first
```

```
model.add(layers.Input(shape=(10000,)))
```

```
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

```
from tensorflow.keras import optimizers
from tensorflow.keras import losses
from tensorflow.keras import metrics
```

```
model.compile(optimizer= optimizers.RMSprop(learning_rate= 0.001),
              loss = losses.binary_crossentropy,
              metrics= [metrics.binary_accuracy])
```

```

xval = xtrain[:10000]
partial_xtrain = xtrain[10000:]

yval = ytrain[:10000]
partial_ytrain = ytrain[10000:]

history = model.fit(partial_xtrain,
                    partial_ytrain,
                    epochs=20,
                    batch_size= 512,
                    validation_data=(xval, yval))

Epoch 1/20
30/30 ----- 5s 97ms/step - binary_accuracy:
0.7754 - loss: 0.5383 - val_binary_accuracy: 0.8575 - val_loss: 0.4145
30/30 ----- 1s 15ms/step - binary_accuracy:
0.9955 - loss: 0.0258 - val_binary_accuracy: 0.8675 - val_loss: 0.5107
Epoch 18/20
30/30 ----- 1s 16ms/step - binary_accuracy:
0.9973 - loss: 0.0202 - val_binary_accuracy: 0.8700 - val_loss: 0.5140
Epoch 19/20
30/30 ----- 1s 17ms/step - binary_accuracy:
0.9972 - loss: 0.0190 - val_binary_accuracy: 0.8642 - val_loss: 0.5765
Epoch 20/20
30/30 ----- 1s 20ms/step - binary_accuracy:
0.9981 - loss: 0.0155 - val_binary_accuracy: 0.8686 - val_loss: 0.5662

history_dict = history.history
history_dict.keys()

dict_keys(['binary_accuracy', 'loss', 'val_binary_accuracy', 'val_loss'])

import matplotlib.pyplot as plt
%matplotlib inline

loss_values = history_dict['loss']
val_loss_values = history_dict['val_loss']

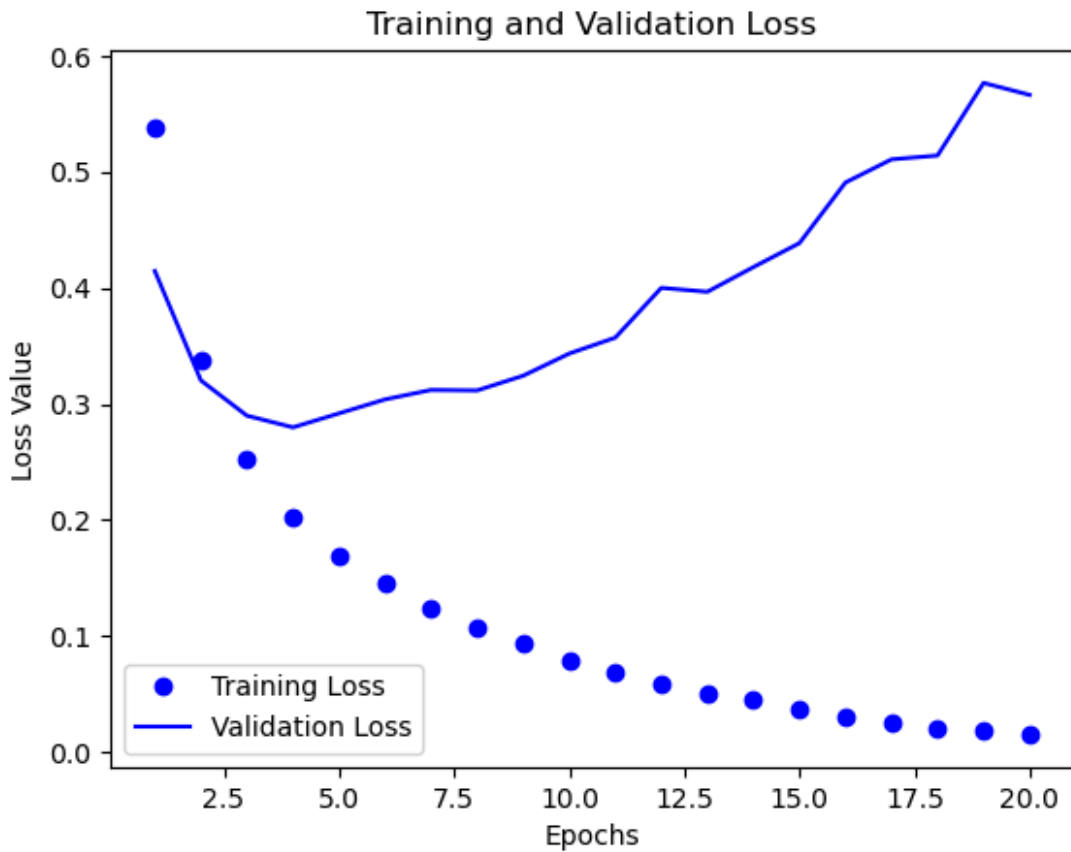
epochs = range(1, len(loss_values)+1)

plt.plot(epochs, loss_values, 'bo', label = "Training Loss")
plt.plot(epochs, val_loss_values, 'b', label = "Validation Loss")

plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss Value')
plt.legend()

plt.show()

```



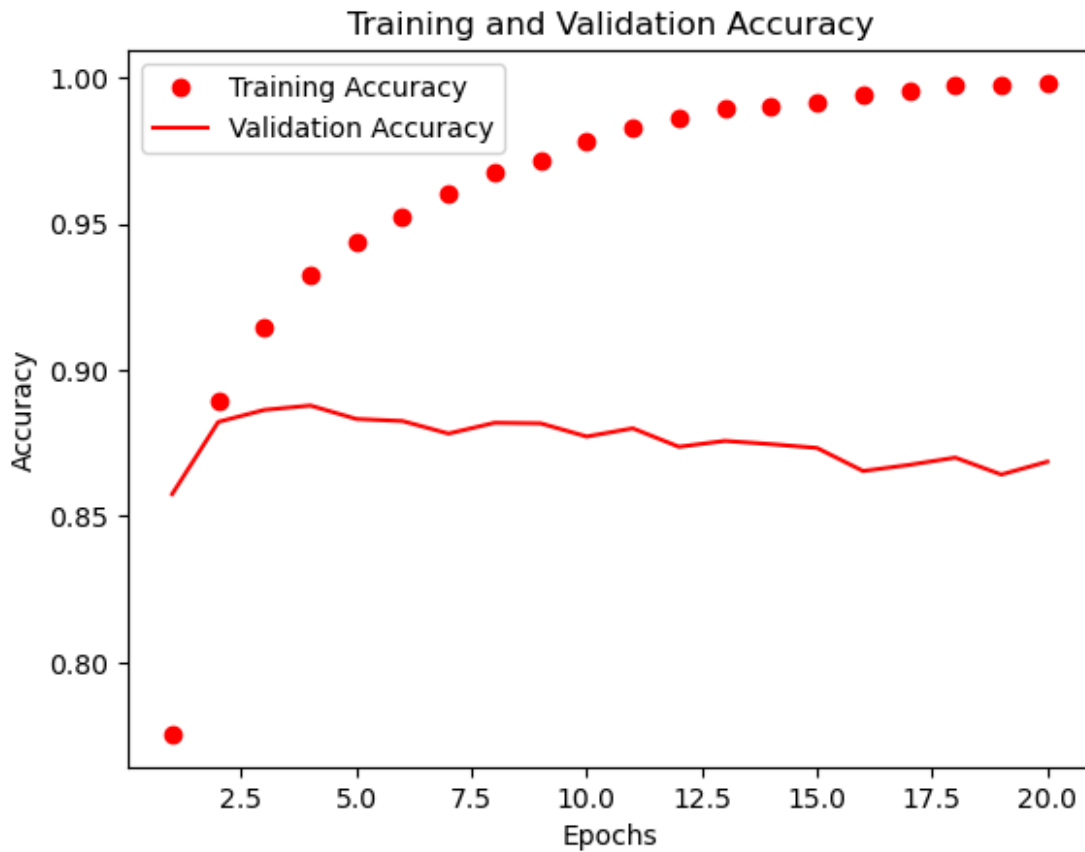
```
acc_values = history_dict['binary_accuracy']
val_acc_values = history_dict['val_binary_accuracy']

epochs = range(1, len(loss_values)+1)

plt.plot(epochs, acc_values, 'ro', label = "Training Accuracy")
plt.plot(epochs, val_acc_values, 'r', label = "Validation Accuracy")

plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

plt.show()
```



```
model.fit(partial_xtrain,
          partial_ytrain,
          epochs= 3,
          batch_size= 512,
          validation_data=(xval,yval))
```

Epoch 1/3

```
30/30 ————— 2s 62ms/step - binary_accuracy:
0.9978 - loss: 0.0139 - val_binary_accuracy: 0.8644 - val_loss: 0.6094
```

Epoch 2/3

```
30/30 ————— 1s 14ms/step - binary_accuracy:
0.9998 - loss: 0.0079 - val_binary_accuracy: 0.8667 - val_loss: 0.6111
```

Epoch 3/3

```
30/30 ————— 1s 17ms/step - binary_accuracy:
0.9984 - loss: 0.0109 - val_binary_accuracy: 0.8675 - val_loss: 0.6290
```

<keras.src.callbacks.history.History at 0x22a6f680b90>

```
np.set_printoptions(suppress= True)
result = model.predict(xtest)
```

```
782/782 ————— 2s 2ms/step
```

```
result
array([[0.01040607],
       [0.9999995 ],
       [0.16961706],
       ...,
       [0.00079786],
       [0.01386299],
       [0.969128  ]], shape=(25000, 1), dtype=float32)
```

```
y_pred = np.zeros(len(result))
for i, score in enumerate(result):
    y_pred[i] = 1 if score > 0.5 else 0

from sklearn.metrics import mean_absolute_error
mae = mean_absolute_error(y_pred, ytest)
```

```
mae
```

```
0.14376
```